NAJJA

Najja Editor v1.6.0 · Penjelasan teknis

Kenapa impor dan play video sekarang lebih cepat

Dokumen ini menjelaskan apa yang membuat Najja Editor terasa lambat sebelumnya, apa yang diubah di backend (server VPS) dan di browser, dan berapa besar perbedaannya menurut pengukuran.

13 November 2026 · Versi 1.6.0 (commit 2a11ff6) · Diukur dengan proyek "NAJJA - 1" (video iPhone HEVC 1080×1920, 47 detik, 63 MB) · Semua data proyek dan media tetap di VPS milikmu.

Ringkasan dalam satu menit

Sebelumnya, editor memutar **video asli** (atau salinan 1080p yang hampir sama beratnya) di pratinjau. Akibatnya perangkat harus mengunduh, men-decode, dan menggambar file besar. Impor juga **menunggu** thumbnail dan waveform dibuat di perangkat.

Sekarang server menyiapkan **salinan ringan 720p** khusus untuk pratinjau, membuat thumbnail dan waveform sendiri, dan impor langsung bisa dipakai. Pemutar hanya menggambar ulang saat videonya punya frame baru. **Hasil ekspor tetap kualitas penuh**, karena ekspor membaca file asli atau salinan 1080p.

0,1 dtk klip muncul di timeline setelah impor sebelumnya 3,2 dtk	**13** dtk video iPhone bisa diputar di PC yang tidak mendukung HEVC sebelumnya 32 dtk	**14** MB yang diunduh perangkat untuk pratinjau video ini sebelumnya 41 MB	**57%** beban browser saat play (ada ruang napas) sebelumnya ±99% (penuh)

Angka konversi di atas diukur dengan server 4 core. Untuk VPS 1 core, lihat bagian 4.

1 Dulu lambatnya di mana?

Lambatnya tidak ditebak, tapi diukur: waktu tiap langkah impor, waktu menggambar tiap frame, dan profil CPU saat video diputar. Ada empat penyebab utama:

① Pratinjau memutar file yang terlalu berat

Salinan pratinjau dibuat 1080p kualitas tinggi: 41 MB, hampir sebesar aslinya (63 MB). Menggambar satu frame 1080p ke layar makan **9,9 ms**, satu frame 720p hanya **5,0 ms**. Dengan filter, angkanya 13,7 ms lawan 7,7 ms.

→ Sekarang: salinan 720p khusus pratinjau (14 MB)

② Impor menunggu pekerjaan berat di perangkat

Sebelum klip bisa dipakai, browser melompat 32 kali di dalam video untuk membuat thumbnail, lalu memuat **seluruh file** ke memori untuk waveform. Di iPad/HP ini lambat, dan untuk file besar bisa membuat tab dimuat ulang.

→ Sekarang: klip langsung dipakai, server yang membuat thumbnail & waveform

③ Video iPhone harus dikonversi lama dulu

Banyak PC tidak bisa memutar HEVC. Server dulu mengonversi ke 1080p kualitas tinggi: **85 detik** untuk video 47 detik di 1 core CPU. Baru sesudah itu video bisa diputar.

→ Sekarang: salinan 720p dibuat duluan (±36 dtk di 1 core, 11 dtk di 4 core)

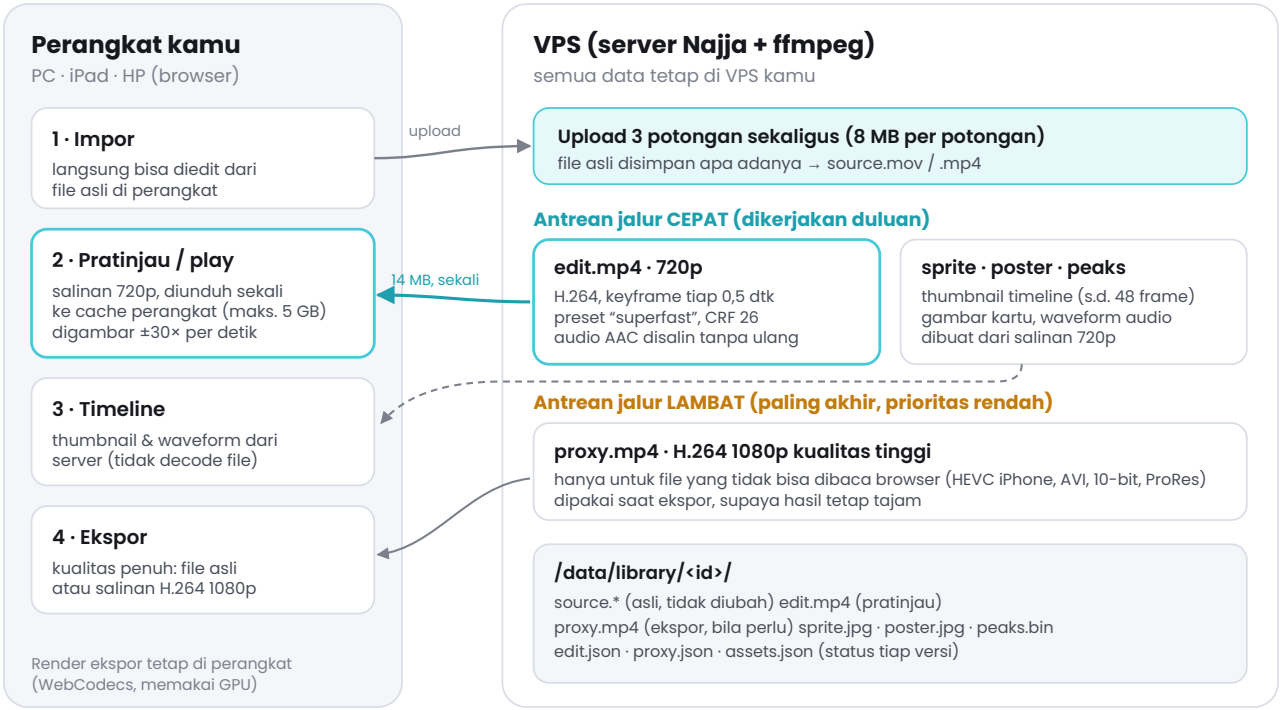
④ Layar digambar ulang terlalu sering

Pratinjau digambar ulang setiap refresh layar: 60× per detik di laptop, 120× di iPad Pro. Padahal videonya cuma 30 fps, jadi separuh atau tiga perempat kerjanya terbuang. Profil CPU: browser sibuk **±99%**, dan 67% dari waktunya habis untuk menggambar frame (drawImage).

→ Sekarang: gambar ulang hanya saat ada frame video baru

2 Cara kerja backend sekarang

Tiap video yang diimpor punya beberapa "versi" di VPS. Masing-masing dipakai untuk tujuan berbeda, dan semuanya dibuat otomatis oleh **ffmpeg** di server:



Satu pekerjaan ffmpeg berjalan pada satu waktu, supaya VPS kecil tidak kehabisan memori dan aplikasi lain di VPS tidak terganggu. Pekerjaan yang ditunggu orang (salinan pratinjau, thumbnail) selalu didahulukan.

Kenapa 720p dan bukan 1080p untuk pratinjau?

- Kotak pratinjau di layar iPad/HP biasanya hanya ±340–700 piksel lebarnya. Detail di atas 720p tidak kelihatan, tapi tetap harus dibayar dengan unduhan, decode, dan waktu menggambar.
- Biaya menggambar terutama tergantung **ukuran video sumber**, bukan ukuran kotak pratinjau. Diukur pada kotak pratinjau yang sama: 1080p butuh 9,9 ms per frame, 720p 5,0 ms.
- **Keyframe tiap 0,5 detik** membuat geser timeline dan lompat waktu hampir instan: browser paling jauh men-decode 15 frame untuk sampai ke posisi mana pun.
- Preset `superfast` dibuat 2,3× lebih cepat dari salinan 1080p lama (36 vs 85 detik di 1 core). Kualitasnya cukup untuk pratinjau, dan memang tidak dipakai untuk hasil akhir.

Versi mana yang dipakai kapan

Situasi	Pratinjau memutar	Ekspor membaca
Baru impor, di perangkat yang mengimpor	file asli dari perangkat (langsung, tanpa unduh)	file asli dari perangkat
Salinan 720p sudah jadi	salinan 720p (diunduh sekali, lalu dari cache perangkat)	file asli, atau salinan 1080p bila asli tidak terbaca browser
Video iPhone (HEVC) di PC yang tidak mendukung HEVC	salinan 720p, segera setelah jadi	salinan H.264 1080p. Kalau belum jadi, ekspor **menunggu** dan tidak diam-diam memakai 720p
Video ringan (≤ 720p dan bitrate ≤ 8 Mbps, H.264)	file asli (tidak perlu salinan)	file asli
Dibuka di perangkat lain / setelah refresh	salinan 720p dari cache perangkat; tidak diunduh ulang	diunduh sekali saat ekspor pertama

3 Perubahan di sisi browser

Impor instan

1. Browser cukup membaca **header** file (durasi, ukuran). Klip langsung muncul di timeline.
2. Upload ke VPS mulai saat itu juga, **3 potongan sekaligus**. Ke server yang jauh, ini lebih cepat daripada satu per satu.
3. Gambar kartu dan thumbnail sementara (16 frame) dibuat di belakang layar, tanpa membuatmu menunggu.
4. Waveform video dan thumbnail lengkap (hingga 48 frame) dibuat **server**. Browser tidak lagi memuat seluruh file ke memori, yang dulu bisa membuat iPad memuat ulang tab.
5. Begitu salinan 720p siap, pemutar pindah otomatis setelah salinannya selesai diunduh, jadi tidak pernah layar kosong.

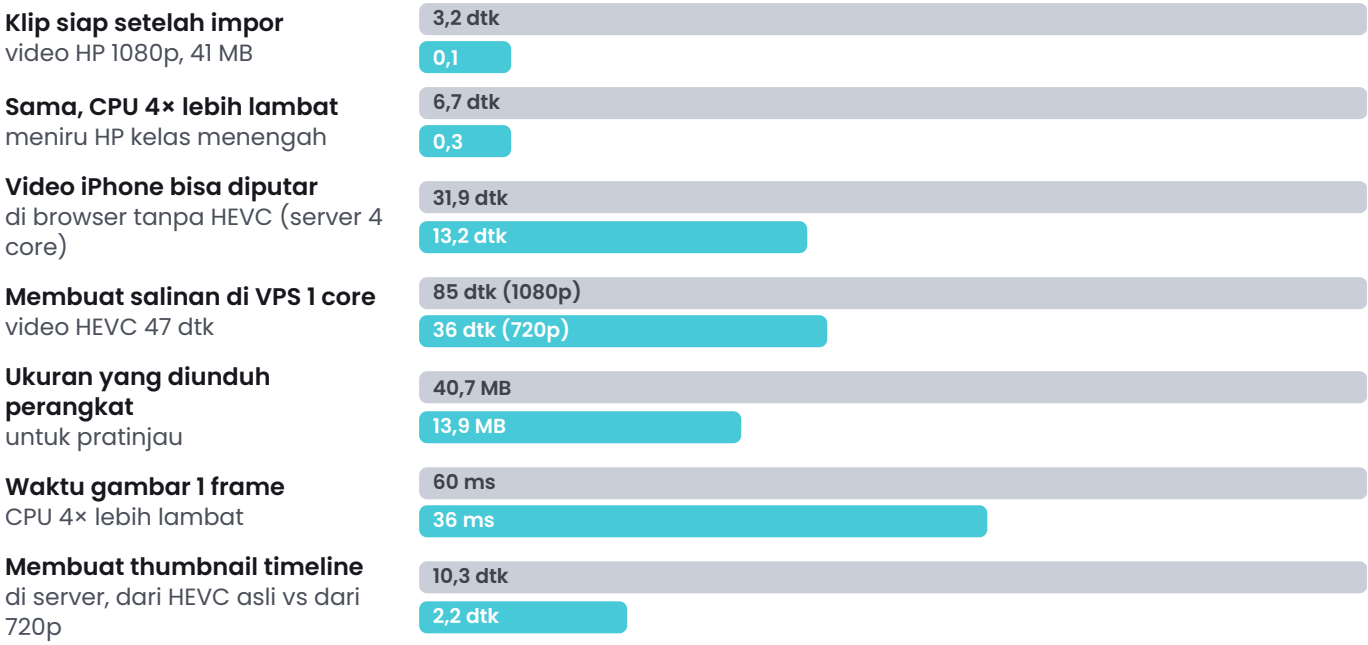
Pemutaran hemat

1. **Gambar ulang mengikuti video.** Pemutar menghitung frame yang benar-benar ditampilkan video (`requestVideoFrameCallback`) dan baru menggambar ulang saat ada frame baru (±30× per detik). Klip tanpa video (teks, foto) mengikuti fps proyek.
2. **Resolusi pratinjau maksimal 1280** di sisi terpanjang. Pratinjau tidak butuh lebih dari itu.
3. **Turun otomatis di perangkat lambat.** Kalau satu frame butuh lebih dari 16 ms, resolusi pratinjau turun bertahap sampai setengahnya. Kalau perangkat longgar lagi, resolusi naik kembali. Ekspor selalu resolusi penuh.
4. **Cache perangkat per versi.** Salinan pratinjau dan salinan ekspor disimpan terpisah di penyimpanan browser (OPFS, maks. 5 GB), jadi menyimpan yang satu tidak menghapus yang lain.

> **Kenapa tidak ganti framework (mis. Tailwind) atau rombak total?** Tailwind hanya mengatur tampilan (CSS) dan tidak menyentuh pengolahan video. Pengukuran menunjukkan 67% waktu browser habis untuk menggambar video yang terlalu besar, dan impor lambat karena menunggu pekerjaan yang sebenarnya bisa dikerjakan server. Jadi yang dirombak adalah pipa media dan mesin pemutar, bagian yang benar-benar jadi penghambat.

4 Hasil pengukuran

v1.5 (sebelum) v1.6 (sesudah) · bar lebih pendek = lebih baik



Pemutaran, angka lengkap

Saat video diputar 6 detik	v1.5	v1.6	Artinya
Gambar ulang per detik	53	**30**	tidak ada kerja terbuang; sesuai fps video
Waktu gambar 1 frame (rata-rata)	13,7 ms	**10,6 ms**	video sumber lebih kecil
Jeda antar-frame layar (95% terburuk)	33,3 ms	**16,7 ms**	tidak ada frame yang tersendat
Frame tersendat (> 50 ms)	2	**0**	
Browser sibuk (profil CPU)	±99%	**±57%**	ada ruang untuk scroll timeline, sentuhan, dll.
CPU 4× lebih lambat: waktu gambar 1 frame	60 ms	**36 ms**	resolusi pratinjau turun otomatis
CPU 4× lebih lambat: frame tersendat	69	42	membaik, tapi HP lemah masih terbatas (lihat bagian 5)
Ekspor MP4 proyek yang sama	67 dtk	68 dtk	sama; kualitas ekspor tidak turun

Cara mengukur

- Browser: Chromium (otomatis lewat Playwright) **tanpa kartu grafis**, jadi semua gambar dikerjakan CPU. Ini kondisi terburuk. Di iPad, laptop, dan HP asli, menggambar video dibantu GPU dan seharusnya lebih ringan lagi.
- Perangkat lambat ditiru dengan memperlambat CPU 4× (fitur bawaan Chrome DevTools).
- Server diuji dengan 4 core. Angka 1 core diukur dengan membatasi ffmpeg ke satu core (`taskset`).
- Versi lama dan baru dijalankan berdampingan pada salinan data proyek yang sama, lewat langkah UI yang sama (tombol + → Impor dari perangkat).
- Upload diuji lokal, jadi waktu upload ke VPS sungguhan tergantung kecepatan internet. Di perangkat yang mengimpor, klip sudah bisa diedit sebelum upload selesai.

5 Yang perlu diketahui

> **Belum diuji di iPad atau HP asli.** Semua angka di dokumen ini dari browser tanpa GPU. Arah perubahannya jelas (lebih sedikit data, lebih sedikit kerja), tapi rasa lancarnya perlu dicoba langsung di perangkatmu.

- **Video lama ikut diproses.** Saat proyek lama dibuka pertama kali sesudah update, server membuat salinan 720p, thumbnail, dan waveform untuk video-videonya, satu per satu. Selama itu VPS sibuk di latar belakang dan pratinjau memakai versi lama dulu.
- **Upload tetap tergantung internet.** Video 1 GB tetap butuh waktu untuk naik ke VPS. Bedanya, sekarang kamu bisa langsung mengedit dari perangkat yang mengimpor.
- **Ruang disk VPS.** Tiap video berat mendapat salinan 720p tambahan, biasanya 15–30% dari ukuran aslinya (contoh: 14 MB untuk file 63 MB).
- **HP yang sangat lemah** masih bisa tersendat di proyek dengan banyak lapisan (filter, efek, teks beranimasi). Pratinjau menurunkan resolusinya sendiri untuk membantu.
- **Ekspor tidak lebih cepat** di versi ini. Kecepatan ekspor sudah dioptimalkan di v1.3 (WebCodecs + GPU).

Langkah berikutnya yang mungkin

- **Penggambar berbasis GPU (WebGL)** untuk filter dan efek. Filter warna akan hampir gratis di semua perangkat.
- **Salinan 720p dibuat di perangkat** (WebCodecs) saat browser mampu, supaya tidak menunggu upload dan VPS.
- **Pengaturan kualitas pratinjau manual** (Hemat / Normal / Tinggi) untuk yang ingin memilih sendiri.

Pengaturan server (opsional, file `.env` di VPS)

Variabel	Default	Fungsi
`NAJJA_EDIT_MAX`	1280	sisi terpanjang salinan pratinjau, dalam piksel
`NAJJA_EDIT_THREADS`	jumlah core (maks. 4)	thread ffmpeg untuk salinan pratinjau
`NAJJA_PROXY_THREADS`	2	thread ffmpeg untuk salinan 1080p (hemat memori)
`NAJJA_PROXY`	(kosong)	isi `0` untuk mematikan semua konversi

File yang berubah di v1.6.0

File	Isi perubahan
`lib/media-proxy.js`	salinan pratinjau 720p, thumbnail/poster/waveform dengan ffmpeg, antrean cepat & lambat
`server.js`	alamat baru untuk salinan pratinjau, thumbnail, poster; status tiap versi
`public/js/media.js`	impor instan, memilih sumber pratinjau dan sumber ekspor, cache per versi
`public/js/engine.js` , `player.js`	gambar ulang mengikuti frame video, resolusi pratinjau adaptif
`public/js/remote.js`	upload 3 potongan paralel
`public/js/fastexport.js` , `exporter.js`	ekspor membaca sumber kualitas penuh